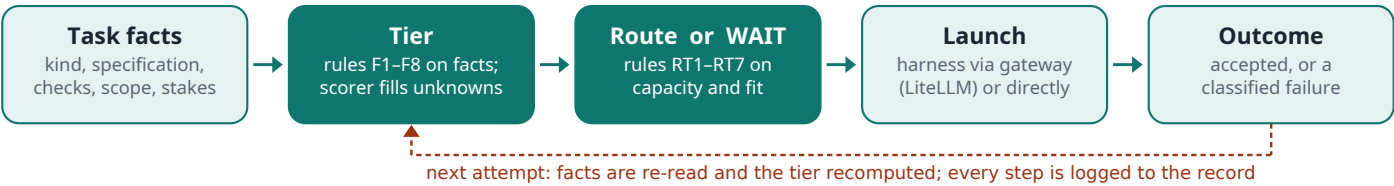


For each unit of work, the router decides which model runs it. It computes the tier the work needs from checkable facts about the task, then picks a model with capacity in that tier. Every decision is deterministic, explainable, logged to the hive record, and recomputed at each attempt. It works for hives on API keys, subscriptions, local models, or a mix.



Deciding the tier

Tiers: local · light · standard · strong. The tier is the highest one any rule requires:

F1	Kind: digest local; work, review light; plan, triage standard; consult strong.
F2	Only a goal, no plan: standard, and suggest splitting off a plan task.
F3	Touches many components: standard.
F4	No check at all: standard.
F5	Costly or hard to undo, without an independent check: strong.
F6	Many tasks blocked on it, without an independent check: strong.
F7	Reviews: at least the author's tier.
F8	Failure history: e.g. a second failed check moves one tier up.

- **Facts, not verdicts.** They come from the task and the record: is there a plan, is there a check the worker can't edit, how much does it touch, what did earlier attempts do.
- **Cheaper means better-specified.** The creator's hint can only raise the tier. To go cheaper, add an independent check, write the plan first, or split the task.
- **Scorer.** A small fixed model estimates *unknown* facts only, through the same rules. Shadow mode first; live once replay shows it helps.
- **Learning.** Replay computes success per kind, facts and tier; a person approves each table change.

Choosing the route

RT1	Operator override wins (recorded).
RT2	Only qualified routes that fit: pool has capacity, cost fits budget, context and tools fit.
RT3	The tier is a floor . Nothing fits → WAIT, never a silent downgrade.
RT4	Reviews use a different model family.
RT5	After a failure, act on its class (right).
RT6	Within the tier: listed order, lowest cost, or most headroom.
RT7	A restart after a crash keeps the same route.

Core ideas

- **Route:** a named, pinned model config; any change makes a new version, so switches are visible. Each passes a **canary** on past tasks, with a recorded lesson, before use.
- **Pool:** capacity with limits: **metered** (API money), **subscription** (usage windows), **local** (concurrency).
- **One route per attempt; scripts, not models**, for polling, parsing and tests.

Failures: one remedy per class

Class	Remedy
outage	Another route in the same tier, or wait. Never a tier up.
capacity	Rate limit, window or budget hit: another pool, or wait.
truncated	Larger limits, or split the task. Not a tier up.
missing info	Get the information, retry on the same route.
failed check	Retry once with feedback, then a tier up or a consult.
stalled	Supervisor or operator decides; a tier up is one option.

Patterns the rules produce

- **Plan strong, execute light.** A plan task yields a plan and checks; the work tasks are then well specified and checked, so they run light.
- **Consult.** A strong model answers one narrow question, with no tools; a light worker does the work.

Gateway: LiteLLM by default, optional

- Routes are gateway aliases; agents get scoped per-actor keys, never provider credentials.
- Usage is measured per actor, task and route, not self-reported. Budgets cap API keys.
- The route table is the source of truth; gateway fallbacks and hidden retries are off.

How it fits One Hive

R1 record	Logs route.* events: decided (with facts), scored, waiting, canary, drift, override, mode.
R5 review	Rules F7 and RT4.
R6 runtime	Asks the router at each attempt start; launches the harness with the answer.
R9 replay	Replays under another table; evaluates the scorer; proposes tuning.

Rollout, and what "done" means

Rules → gateway → usage tracking → **fixed** mode baseline → canaries → **live** → scorer in shadow → record events → tuning.
Done: a hive uses it for real work; more accepted tasks per unit of capacity than the baseline, with no drop in review pass rate.
Not in v1: the scorer setting tiers directly, automatic tuning, a spend ledger, mid-attempt switches.